Flower_Classification_2025

June 9, 2025

1 Flower Classification Competition (30%)

For this competition, we will use the Flower Recognition. This dataset contains 4317 images of flowers. The data collection is based on the data from flickr, google images, yandex images. The aim is to recognize flower species from a photo.

The pictures are divided into five classes: chamomile, tulip, rose, sunflower, dandelion. For each class there are about 800 photos. Photos are not high resolution, about 320x240 pixels. Note that for this dataset the photos are not all at a fixed size, they have different proportions.

We provide baseline code that includes the following features:

- Loading and Analysing the Flowers dataset using torchvision.
- Providing some augmentations (on loading)
- Defining a simple convolutional neural network.
- How to use existing loss function for the model learning.
- Train the network on the training data.
- Test the trained network on the testing data.
- Generate prediction for the random test image(s).

1.1 The following improvements could be considered:

1. Change of advanced training parameters: Learning Rate, Optimizer, Batch-size, Number of Max Epochs, and Drop-out.
2. Use of a new loss function.
3. Additional/better data augmentation
4. Architectural Changes: Batch Normalization, Residual layers, Attention Block, and other variants.

Your code should be modified from the provided baseline. A pdf report of a maximum of two pages is required to explain the changes you made from the baseline, why you chose those changes, and the improvements they achieved.

1.2 Marking Rules:

We will mark the competition based on the final test accuracy on testing images and your report.

Final mark (out of 50) = acc_mark + efficiency mark + report mark ###Acc_mark 10:

We will rank all the submission results based on their test accuracy. Zero improvement over the baseline yields 0 marks. Maximum improvement over the baseline will yield 10 marks. There will be a sliding scale applied in between.

###Efficiency mark 10:

Efficiency considers not only the accuracy, but the computational cost of running the model (flops: https://en.wikipedia.org/wiki/FLOPS). Efficiency for our purposes is defined to be the ratio of accuracy (in %) to Gflops. Please report the computational cost for your final model and include the efficiency calculation in your report. Maximum improvement over the baseline will yield 10 marks. Zero improvement over the baseline yields zero marks, with a sliding scale in between.

###Report mark 30: Your report should comprise: 1. An introduction showing your understanding of the task and of the baseline model: [10 marks]

2. A description of how you have modified aspects of the system to improve performance. [10 marks]

A recommended way to present a summary of this is via an "ablation study" table, eg:

Method1	Method2	Method3	Accuracy
N	N	N	60%
Y	N	N	65%
Y	Y	N	77%
Y	Y	Y	82%

3. Explanation of the methods for reducing the computational cost and/or improve the trade-off between accuracy and cost: [5 marks]

4. Limitations/Conclusions: [5 marks]

[ ]:
```
##########################################################################################
### Subject: Computer Vision
### Year: 2025
### Student Name: Liam Blaschka
### Student ID: a1888420
### Comptetion Name: Flowers Classification Competition
### Final Results:
### ACC: 92.593%        GFLOPs: 0.42060
##########################################################################################
```

[2]:
```
# Importing libraries.

import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import random

# To avoid non-essential warnings
import warnings
```

```
warnings.filterwarnings('ignore')

%matplotlib inline
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
from torchvision.utils import make_grid
from sklearn.model_selection import train_test_split
```

[3]:
```
# Performing Image Transformations.
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),              # instead of Resize +␣
 ↪CenterCrop
    transforms.RandomHorizontalFlip(),              # add randomness
    transforms.ColorJitter(brightness=0.2, contrast=0.2),  # small color shifts
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],       # same normalization
                         [0.229, 0.224, 0.225])
])
```

[4]:
```
# Mounting G-Drive to get your dataset.
# To access Google Colab GPU; Go To: Edit >>> Network Settings >>> Hardware␣
 ↪Accelarator: Select GPU.
# Reference: https://towardsdatascience.com/
 ↪google-colab-import-and-export-datasets-eccf801e2971
from google.colab import drive
drive.mount('/content/drive')

# Dataset path. Ensure that the file path correspond to the path you have here.␣
 ↪It is expected that you unzip the data folders before running the notebook.
data_directory = '/content/drive/MyDrive/Datasets/flower/flowers'
dataset=datasets.ImageFolder(root=data_directory,transform=train_transform)
dataset
```

```
Mounted at /content/drive
```

[4]:
```
Dataset ImageFolder
    Number of datapoints: 4317
    Root location: /content/drive/MyDrive/Datasets/flower/flowers
    StandardTransform
Transform: Compose(
               Resize(size=224, interpolation=bilinear, max_size=None,
antialias=True)
               CenterCrop(size=(224, 224))
               ToTensor()
               Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
           )
```

[4]:
```python
# Run locally
data_directory = './flower/flowers'
dataset=datasets.ImageFolder(root=data_directory,transform=train_transform)
dataset
```

[4]:
```
Dataset ImageFolder
    Number of datapoints: 4317
    Root location: ./flower/flowers
    StandardTransform
Transform: Compose(
               RandomResizedCrop(size=(224, 224), scale=(0.08, 1.0),
ratio=(0.75, 1.3333), interpolation=bilinear, antialias=True)
               RandomHorizontalFlip(p=0.5)
               ColorJitter(brightness=(0.8, 1.2), contrast=(0.8, 1.2),
saturation=None, hue=None)
               ToTensor()
               Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
           )
```

[5]:
```python
# Checking the flower class types.
class_names=dataset.classes
print(class_names)
print(len(class_names))
```

```
['daisy', 'dandelion', 'rose', 'sunflower', 'tulip']
5
```

[6]:
```python
# Train and Test data split.
train_indices, test_indices = train_test_split(list(range(len(dataset.
 ↪targets))), test_size=0.2, stratify=dataset.targets)
train_data = torch.utils.data.Subset(dataset, train_indices)
test_data = torch.utils.data.Subset(dataset, test_indices)
```

[7]:
```python
# To check wether Google Colab GPU has been assigned/not.
torch.cuda.is_available()
def get_default_device():
    """Pick GPU if available, else CPU"""
    if torch.cuda.is_available():
        return torch.device('cuda')
    else:
        return None
device = get_default_device()
device
```

[7]:
```
device(type='cuda')
```

[8]:
```python
def to_device(data, device):
    """Move tensor(s) to chosen device"""
```

```
    if isinstance(data, (list,tuple)):
        return [to_device(x, device) for x in data]
    return data.to(device, non_blocking=True)

class DeviceDataLoader():
    """Wrap a dataloader to move data to a device"""
    def __init__(self, dl, device):
        self.dl = dl
        self.device = device

    def __iter__(self):
        """Yield a batch of data after moving it to device"""
        for b in self.dl:
            yield to_device(b, self.device)

    def __len__(self):
        """Number of batches"""
        return len(self.dl)
```

[9]:
```
BATCH_SIZE = 64

train_loader=DataLoader(train_data,batch_size=BATCH_SIZE,shuffle=True)
test_loader=DataLoader(test_data,batch_size=BATCH_SIZE)
```

[10]:
```
print(len(train_data))
print(len(test_data))
```

```
3453
864
```

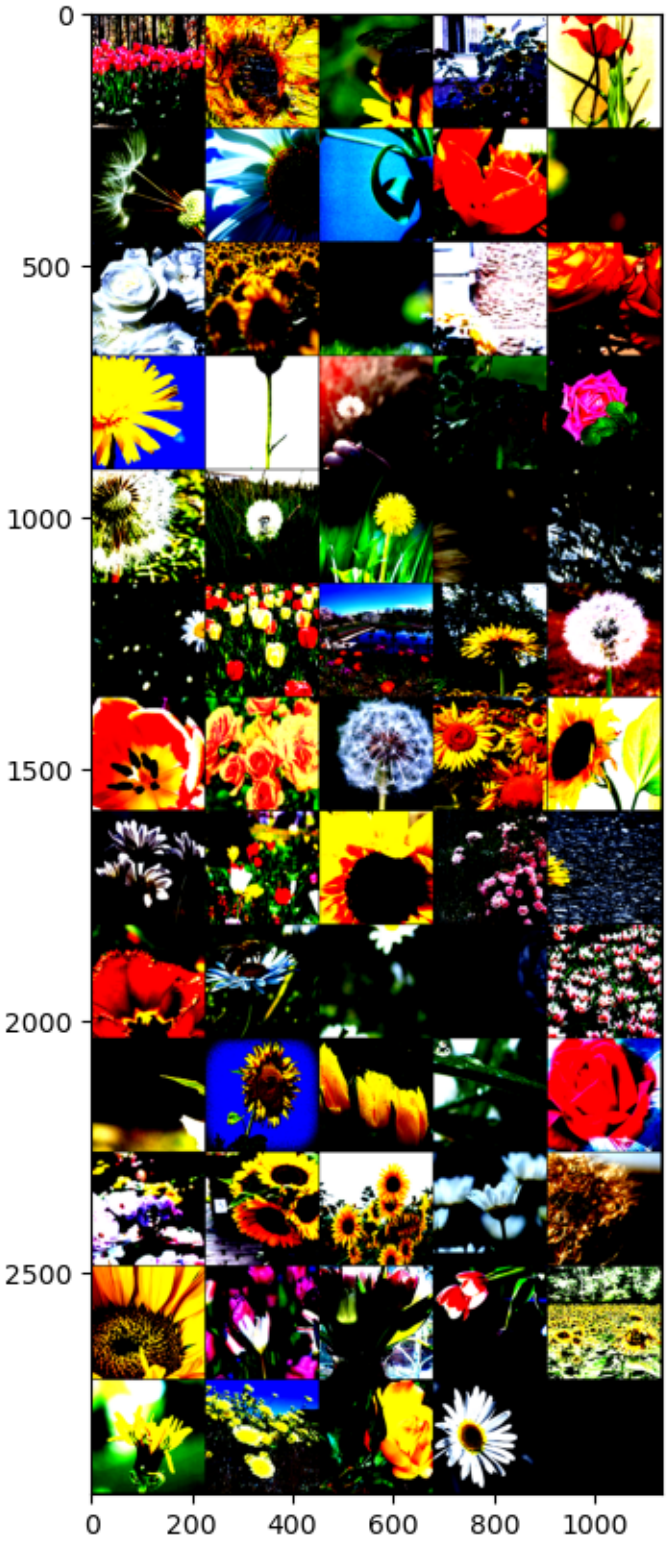
[45]:
```
# Preview of the datasets.
for images, labels in train_loader:
    break
#print the labels
print('Label:', labels.numpy())
print('Class:', *np.array([class_names[i] for i in labels]))

im=make_grid(images,nrow=5)
```

```
Label: [4 3 1 3 4 1 0 4 4 1 2 3 0 2 2 1 0 1 2 2 1 1 1 1 0 0 4 4 1 1 4 2 1 3 3 0
4
 3 2 1 4 1 0 4 4 3 3 4 0 2 2 3 3 0 1 3 4 4 4 3 1 1 2 0]
Class: tulip sunflower dandelion sunflower tulip dandelion daisy tulip tulip
dandelion rose sunflower daisy rose rose dandelion daisy dandelion rose rose
dandelion dandelion dandelion dandelion daisy daisy tulip tulip dandelion
dandelion tulip rose dandelion sunflower sunflower daisy tulip sunflower rose
dandelion tulip dandelion daisy tulip tulip sunflower sunflower tulip daisy rose
rose sunflower sunflower daisy dandelion sunflower tulip tulip tulip sunflower
```

```
dandelion dandelion rose daisy
```

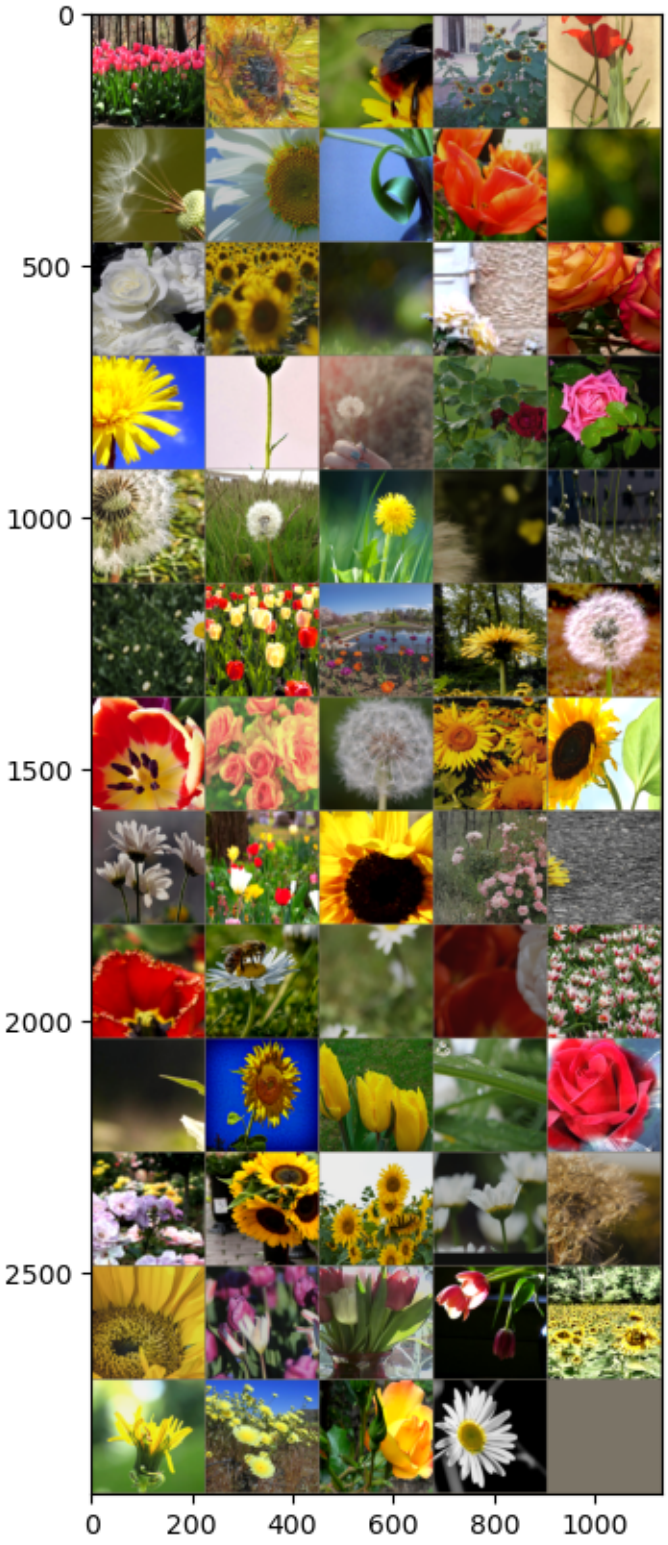
[46]:
```
plt.figure(figsize=(10,10))
plt.imshow(np.transpose(im.numpy(),(1,2,0)))

plt.show()
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for
floats or [0..255] for integers). Got range [-2.117904..2.64].
```


0
500
1000
1500
2000
2500
0
200
400
600
800
1000

[47]:
```
# Inverse Normalization.
inv_normalize=transforms.Normalize(mean=[-0.485/0.229,-0.456/0.224,-0.406/0.
↪225],
                                   std=[1/0.229,1/0.224,1/0.225])
im=inv_normalize(im)
```

[48]:
```
plt.figure(figsize=(10,10))
plt.imshow(np.transpose(im.numpy(),(1,2,0)))

plt.show()
```


0
500
1000
1500
2000
2500
0
200
400
600
800
1000

[ ]:
```
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

class TransferLearningModel(nn.Module):
    def __init__(self, classes):
        super().__init__()

        weights = EfficientNet_B0_Weights.DEFAULT
        self.model = efficientnet_b0(weights=weights)

        self.model.classifier[1] = nn.Linear(self.model.classifier[1].
 ↪in_features, classes)

    def forward(self, x):
        return self.model(x)


LEARNING_RATE = 0.001

num_classes = 5

model = to_device(TransferLearningModel(num_classes), device)


train_loader = DeviceDataLoader(train_loader, device)
test_loader = DeviceDataLoader(test_loader, device)


criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)
```

[126]:
```
model
```

[126]:
```
TransferLearningModel(
  (model): EfficientNet(
    (features): Sequential(
      (0): Conv2dNormActivation(
        (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1),
bias=False)
        (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (2): SiLU(inplace=True)
      )
      (1): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
```

```
              (0): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1), groups=32, bias=False)
              (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(32, 8, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(8, 32, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (2): Conv2dNormActivation(
              (0): Conv2d(32, 16, kernel_size=(1, 1), stride=(1, 1), bias=False)
              (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.0, mode=row)
        )
      )
      (2): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(16, 96, kernel_size=(1, 1), stride=(1, 1), bias=False)
              (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(96, 96, kernel_size=(3, 3), stride=(2, 2), padding=(1,
1), groups=96, bias=False)
              (1): BatchNorm2d(96, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(96, 4, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(4, 96, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(96, 24, kernel_size=(1, 1), stride=(1, 1), bias=False)
```

```
              (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.0125, mode=row)
        )
        (1): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(144, 144, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), groups=144, bias=False)
              (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(144, 6, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(6, 144, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(144, 24, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(24, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.025, mode=row)
        )
      )
      (3): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(24, 144, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(144, 144, kernel_size=(5, 5), stride=(2, 2),
padding=(2, 2), groups=144, bias=False)
              (1): BatchNorm2d(144, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(144, 6, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(6, 144, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(144, 40, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(40, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.037500000000000006, mode=row)
        )
        (1): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(40, 240, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(240, 240, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=240, bias=False)
              (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(240, 10, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(10, 240, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
```

```
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(240, 40, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(40, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.05, mode=row)
        )
      )
      (4): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(40, 240, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(240, 240, kernel_size=(3, 3), stride=(2, 2),
padding=(1, 1), groups=240, bias=False)
              (1): BatchNorm2d(240, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(240, 10, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(10, 240, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(240, 80, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.0625, mode=row)
        )
        (1): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
```

```
              (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(480, 480, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), groups=480, bias=False)
              (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(480, 80, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.07500000000000001, mode=row)
        )
        (2): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(480, 480, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), groups=480, bias=False)
              (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
```

```
              (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(480, 80, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(80, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.08750000000000001, mode=row)
        )
      )
      (5): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(80, 480, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(480, 480, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=480, bias=False)
              (1): BatchNorm2d(480, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(480, 20, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(20, 480, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(480, 112, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.1, mode=row)
```

```
        )
        (1): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=672, bias=False)
              (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(672, 112, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.1125, mode=row)
        )
        (2): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=672, bias=False)
              (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(672, 112, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(112, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.125, mode=row)
        )
      )
      (6): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(112, 672, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(672, 672, kernel_size=(5, 5), stride=(2, 2),
padding=(2, 2), groups=672, bias=False)
              (1): BatchNorm2d(672, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(672, 28, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(28, 672, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(672, 192, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
```

```
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.1375, mode=row)
        )
        (1): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=1152, bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.15000000000000002, mode=row)
        )
        (2): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
```

```
              (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=1152, bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.1625, mode=row)
        )
        (3): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(1152, 1152, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2), groups=1152, bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(1152, 192, kernel_size=(1, 1), stride=(1, 1),
```

```
bias=False)
              (1): BatchNorm2d(192, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.17500000000000002, mode=row)
        )
      )
      (7): Sequential(
        (0): MBConv(
          (block): Sequential(
            (0): Conv2dNormActivation(
              (0): Conv2d(192, 1152, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (1): Conv2dNormActivation(
              (0): Conv2d(1152, 1152, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1), groups=1152, bias=False)
              (1): BatchNorm2d(1152, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
              (2): SiLU(inplace=True)
            )
            (2): SqueezeExcitation(
              (avgpool): AdaptiveAvgPool2d(output_size=1)
              (fc1): Conv2d(1152, 48, kernel_size=(1, 1), stride=(1, 1))
              (fc2): Conv2d(48, 1152, kernel_size=(1, 1), stride=(1, 1))
              (activation): SiLU(inplace=True)
              (scale_activation): Sigmoid()
            )
            (3): Conv2dNormActivation(
              (0): Conv2d(1152, 320, kernel_size=(1, 1), stride=(1, 1),
bias=False)
              (1): BatchNorm2d(320, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
            )
          )
          (stochastic_depth): StochasticDepth(p=0.1875, mode=row)
        )
      )
      (8): Conv2dNormActivation(
        (0): Conv2d(320, 1280, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(1280, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
        (2): SiLU(inplace=True)
```

```
      )
    )
    (avgpool): AdaptiveAvgPool2d(output_size=1)
    (classifier): Sequential(
      (0): Dropout(p=0.2, inplace=True)
      (1): Linear(in_features=1280, out_features=5, bias=True)
    )
  )
)
```

[127]:
```python
# Counting of number of parameters in the model.
def count_parameters(model):
    params = [p.numel() for p in model.parameters() if p.requires_grad]
    for item in params:
        print(f'{item:>8}')
    print(f'________\n{sum(params):>8}')

count_parameters(model)
```

```
     864
      32
      32
     288
      32
      32
     256
       8
     256
      32
     512
      16
      16
    1536
      96
      96
     864
      96
      96
     384
       4
     384
      96
    2304
      24
      24
    3456
     144
     144
```

1296
144
144
864
6
864
144
3456
24
24
3456
144
144
3600
144
144
864
6
864
144
5760
40
40
9600
240
240
6000
240
240
2400
10
2400
240
9600
40
40
9600
240
240
2160
240
240
2400
10
2400
240
19200
80

80
38400
480
480
4320
480
480
9600
20
9600
480
38400
80
80
38400
480
480
4320
480
480
9600
20
9600
480
38400
80
80
38400
480
480
12000
480
480
9600
20
9600
480
53760
112
112
75264
672
672
16800
672
672
18816
28

18816
672
75264
112
112
75264
672
672
16800
672
672
18816
28
18816
672
75264
112
112
75264
672
672
16800
672
672
18816
28
18816
672
129024
192
192
221184
1152
1152
28800
1152
1152
55296
48
55296
1152
221184
192
192
221184
1152
1152
28800

```
     1152
     1152
    55296
       48
    55296
     1152
   221184
      192
      192
   221184
     1152
     1152
    28800
     1152
     1152
    55296
       48
    55296
     1152
   221184
      192
      192
   221184
     1152
     1152
    10368
     1152
     1152
    55296
       48
    55296
     1152
   368640
      320
      320
   409600
     1280
     1280
     6400
        5
--------
 4013953
```

[ ]:
```python
# Learning Schema.
import time
start_time=time.time()
train_losses=[]
```

```
test_losses=[]
train_correct=[]
test_correct=[]
epochs=5

for i in range(epochs):
    trn_corr=0
    tst_corr=0
    for b, (X_train,y_train) in enumerate(train_loader):
        b+=1
        # print(X_train.device,y_train.device)
        y_pred=model(X_train)
        # print(y_pred.device)
        loss=criterion(y_pred,y_train)

        predicted=torch.max(y_pred.data,1)[1]
        batch_corr=(predicted==y_train).sum()
        trn_corr+=batch_corr

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        # Note that this line only gives training accuracy
        if b%10==0:
            print(f"epoch: {i} loss: {loss.item()} batch: {b} accuracy:␣
↪{trn_corr.item()*100/(10*b):7.3f}%")
    loss=loss.detach()
    train_losses.append(loss.cpu().numpy())
    train_correct.append(trn_corr.cpu().numpy())


    with torch.no_grad():
        for b, (X_test,y_test) in enumerate(test_loader):
            y_val=model(X_test)
            loss=criterion(y_val,y_test)

            predicted=torch.max(y_val.data,1)[1]
            btach_corr=(predicted==y_test).sum()
            tst_corr+=btach_corr

        loss=loss.detach().cpu().numpy()
        test_losses.append(loss)
        test_correct.append(tst_corr.cpu().numpy())
    print(f'epoch: {i}, test accuracy: {tst_corr*100/len(test_data):7.3f}%')

print(f'\nDuration: {time.time() - start_time:.0f} seconds')
```

```
epoch: 0 loss: 0.7504527568817139 batch: 10 accuracy: 452.000%
```

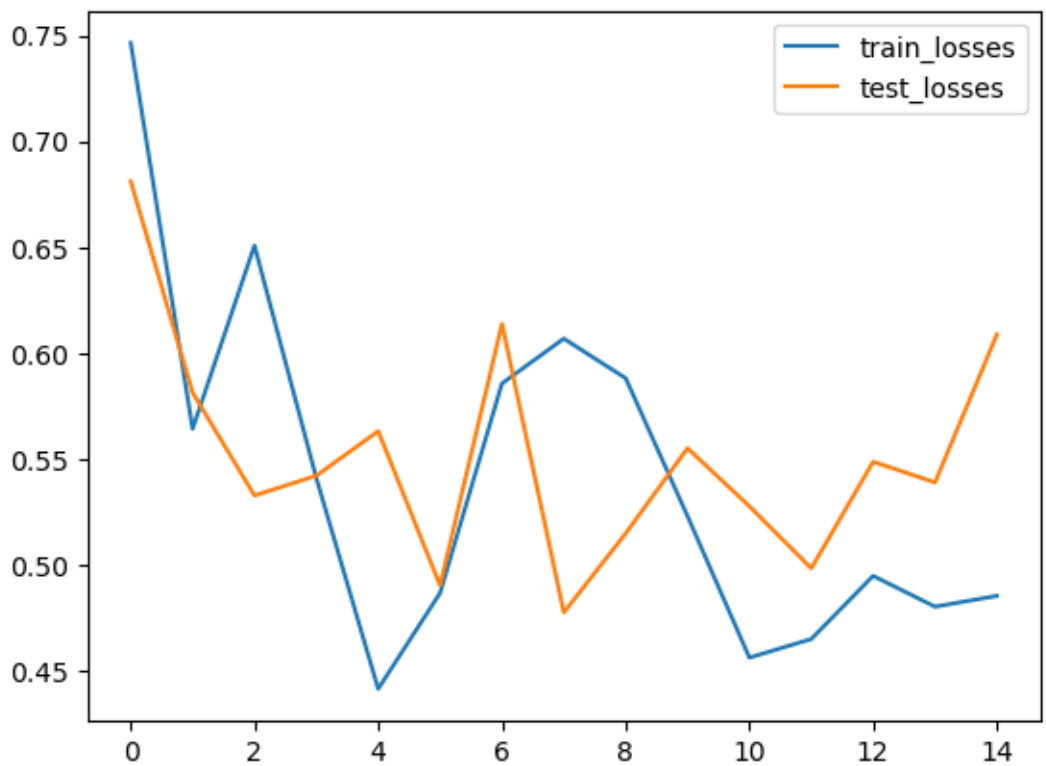
```
epoch: 0 loss: 0.7268800735473633 batch: 20 accuracy: 493.500%
epoch: 0 loss: 0.7342614531517029 batch: 30 accuracy: 503.667%
epoch: 0 loss: 0.6718704700469971 batch: 40 accuracy: 511.500%
epoch: 0 loss: 0.6261208653450012 batch: 50 accuracy: 521.200%
epoch: 0, test accuracy:  86.806%
epoch: 1 loss: 0.6538276672363281 batch: 10 accuracy: 562.000%
epoch: 1 loss: 0.5827730894088745 batch: 20 accuracy: 568.500%
epoch: 1 loss: 0.5673778057098389 batch: 30 accuracy: 570.000%
epoch: 1 loss: 0.6440584659576416 batch: 40 accuracy: 566.750%
epoch: 1 loss: 0.5727642178535461 batch: 50 accuracy: 570.000%
epoch: 1, test accuracy:  88.310%
epoch: 2 loss: 0.48762428760528564 batch: 10 accuracy: 594.000%
epoch: 2 loss: 0.6758631467819214 batch: 20 accuracy: 592.500%
epoch: 2 loss: 0.523586630821228 batch: 30 accuracy: 590.000%
epoch: 2 loss: 0.6108452081680298 batch: 40 accuracy: 586.000%
epoch: 2 loss: 0.6475273370742798 batch: 50 accuracy: 584.600%
epoch: 2, test accuracy:  90.856%
epoch: 3 loss: 0.652725338935852 batch: 10 accuracy: 589.000%
epoch: 3 loss: 0.5633979439735413 batch: 20 accuracy: 593.000%
epoch: 3 loss: 0.6519349217414856 batch: 30 accuracy: 587.000%
epoch: 3 loss: 0.6145844459533691 batch: 40 accuracy: 589.750%
epoch: 3 loss: 0.5959717631340027 batch: 50 accuracy: 588.600%
epoch: 3, test accuracy:  89.468%
epoch: 4 loss: 0.5182311534881592 batch: 10 accuracy: 589.000%
epoch: 4 loss: 0.603959321975708 batch: 20 accuracy: 590.500%
epoch: 4 loss: 0.591626763343811 batch: 30 accuracy: 590.667%
epoch: 4 loss: 0.5685918927192688 batch: 40 accuracy: 591.000%
epoch: 4 loss: 0.563170313835144 batch: 50 accuracy: 593.600%
epoch: 4, test accuracy:  89.815%
epoch: 5 loss: 0.5172511339187622 batch: 10 accuracy: 604.000%
epoch: 5 loss: 0.6040385961532593 batch: 20 accuracy: 601.000%
epoch: 5 loss: 0.5720844268798828 batch: 30 accuracy: 600.000%
epoch: 5 loss: 0.5739210247993469 batch: 40 accuracy: 598.250%
epoch: 5 loss: 0.578086793422699 batch: 50 accuracy: 598.000%
epoch: 5, test accuracy:  90.856%
epoch: 6 loss: 0.5534927248954773 batch: 10 accuracy: 599.000%
epoch: 6 loss: 0.5918945074081421 batch: 20 accuracy: 597.000%
epoch: 6 loss: 0.4785706400871277 batch: 30 accuracy: 599.000%
epoch: 6 loss: 0.7249789237976074 batch: 40 accuracy: 597.250%
epoch: 6 loss: 0.6223516464233398 batch: 50 accuracy: 594.600%
epoch: 6, test accuracy:  89.815%
epoch: 7 loss: 0.4554044306278229 batch: 10 accuracy: 603.000%
epoch: 7 loss: 0.4749644696712494 batch: 20 accuracy: 607.500%
epoch: 7 loss: 0.5154200792312622 batch: 30 accuracy: 603.333%
epoch: 7 loss: 0.45620882511138916 batch: 40 accuracy: 601.500%
epoch: 7 loss: 0.528810977935791 batch: 50 accuracy: 600.400%
epoch: 7, test accuracy:  90.509%
epoch: 8 loss: 0.5773119926452637 batch: 10 accuracy: 593.000%
```

```
epoch: 8 loss: 0.529985785484314 batch: 20 accuracy: 598.000%
epoch: 8 loss: 0.4719991087913513 batch: 30 accuracy: 602.667%
epoch: 8 loss: 0.5725828409194946 batch: 40 accuracy: 600.500%
epoch: 8 loss: 0.46710723638534546 batch: 50 accuracy: 603.200%
epoch: 8, test accuracy:  90.278%
epoch: 9 loss: 0.5090176463127136 batch: 10 accuracy: 612.000%
epoch: 9 loss: 0.5085698366165161 batch: 20 accuracy: 608.000%
epoch: 9 loss: 0.5925135016441345 batch: 30 accuracy: 602.667%
epoch: 9 loss: 0.4961862564086914 batch: 40 accuracy: 602.750%
epoch: 9 loss: 0.5497264862060547 batch: 50 accuracy: 601.800%
epoch: 9, test accuracy:  89.931%
epoch: 10 loss: 0.4603850245475769 batch: 10 accuracy: 611.000%
epoch: 10 loss: 0.5259632468223572 batch: 20 accuracy: 607.500%
epoch: 10 loss: 0.4855652451515198 batch: 30 accuracy: 603.333%
epoch: 10 loss: 0.5654672980308533 batch: 40 accuracy: 605.500%
epoch: 10 loss: 0.43486061692237854 batch: 50 accuracy: 606.200%
epoch: 10, test accuracy:  89.931%
epoch: 11 loss: 0.5432144403457642 batch: 10 accuracy: 609.000%
epoch: 11 loss: 0.5267478227615356 batch: 20 accuracy: 609.500%
epoch: 11 loss: 0.560116708278656 batch: 30 accuracy: 607.000%
epoch: 11 loss: 0.5215336680412292 batch: 40 accuracy: 608.000%
epoch: 11 loss: 0.5097153782844543 batch: 50 accuracy: 607.200%
epoch: 11, test accuracy:  91.319%
epoch: 12 loss: 0.5660734176635742 batch: 10 accuracy: 605.000%
epoch: 12 loss: 0.4768533706665039 batch: 20 accuracy: 606.000%
epoch: 12 loss: 0.44502490758895874 batch: 30 accuracy: 605.667%
epoch: 12 loss: 0.5312442779541016 batch: 40 accuracy: 605.250%
epoch: 12 loss: 0.6038652658462524 batch: 50 accuracy: 602.600%
epoch: 12, test accuracy:  90.046%
epoch: 13 loss: 0.4797636866569519 batch: 10 accuracy: 608.000%
epoch: 13 loss: 0.5486522912979126 batch: 20 accuracy: 610.000%
epoch: 13 loss: 0.5017635822296143 batch: 30 accuracy: 610.667%
epoch: 13 loss: 0.5159293413162231 batch: 40 accuracy: 612.000%
epoch: 13 loss: 0.44417399168014526 batch: 50 accuracy: 613.600%
epoch: 13, test accuracy:  91.667%
epoch: 14 loss: 0.4918074905872345 batch: 10 accuracy: 616.000%
epoch: 14 loss: 0.49965184926986694 batch: 20 accuracy: 617.000%
epoch: 14 loss: 0.4473852515220642 batch: 30 accuracy: 615.333%
epoch: 14 loss: 0.4857326149940491 batch: 40 accuracy: 614.500%
epoch: 14 loss: 0.4370453953742981 batch: 50 accuracy: 613.400%
epoch: 14, test accuracy:  92.593%

Duration: 251 seconds
```

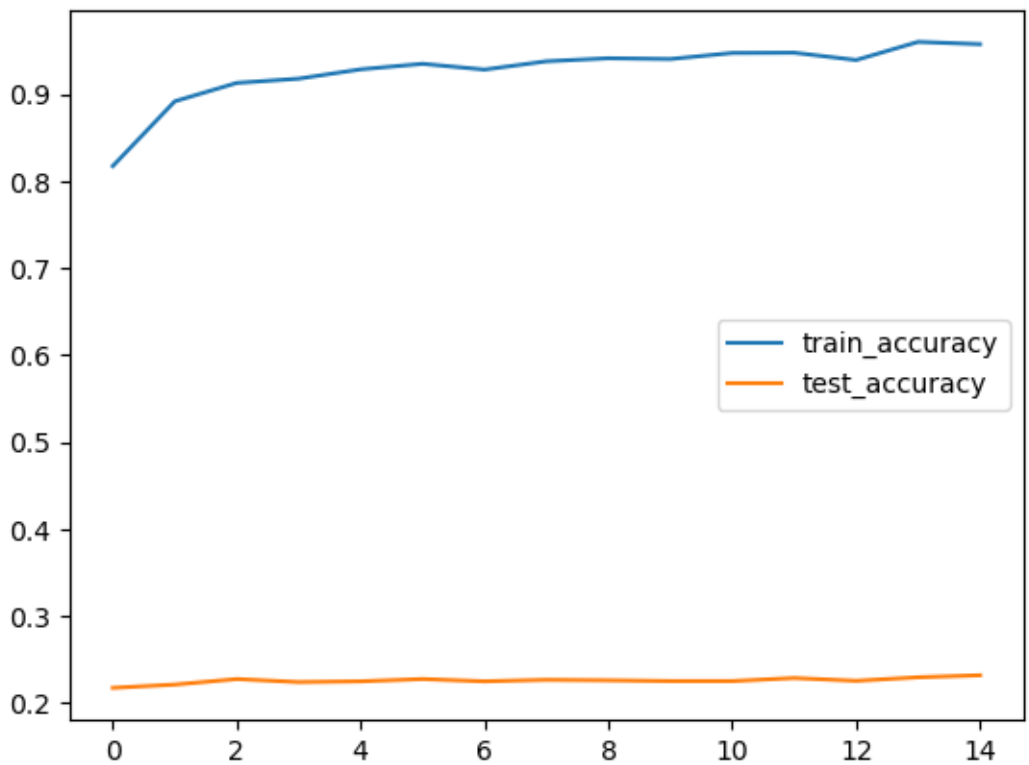
I chose 15 epochs when testing my model as in the output above. After running it, I changed the code back to 5 epochs as it was originally set.

[130]:
```
# Plotting loss over time.
plt.plot(train_losses,label="train_losses")
plt.plot(test_losses,label="test_losses")
plt.legend()
print(train_losses)

plt.show()
```

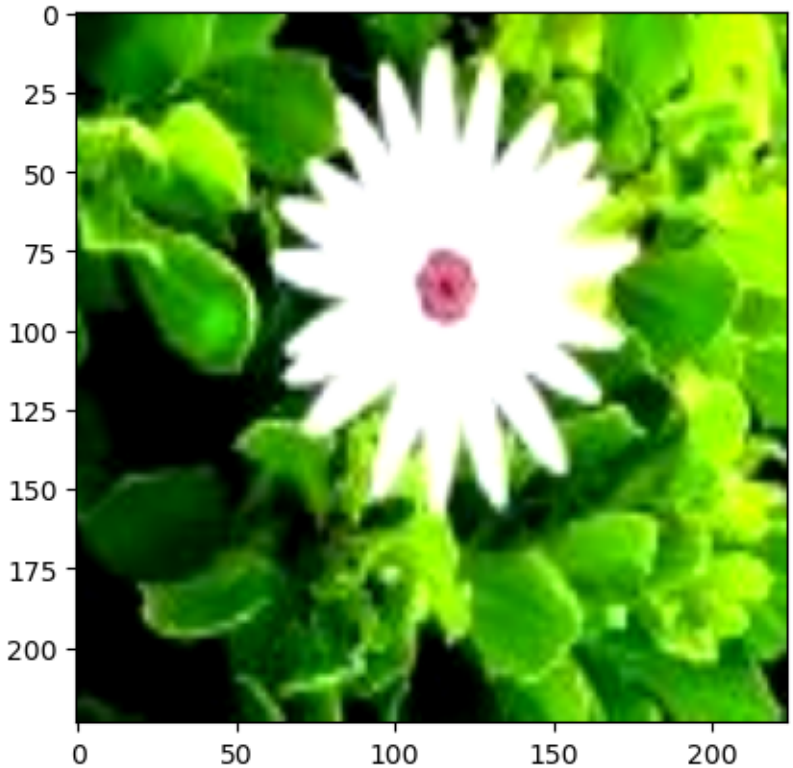
```
[array(0.7464453, dtype=float32), array(0.56420267, dtype=float32),
array(0.65076387, dtype=float32), array(0.5409676, dtype=float32),
array(0.44141957, dtype=float32), array(0.48667797, dtype=float32),
array(0.5856039, dtype=float32), array(0.60680425, dtype=float32),
array(0.5881197, dtype=float32), array(0.5229953, dtype=float32),
array(0.4561638, dtype=float32), array(0.46493256, dtype=float32),
array(0.49477714, dtype=float32), array(0.480233, dtype=float32),
array(0.4853226, dtype=float32)]
```



[131]:
```
plt.plot(np.array(train_correct)/len(train_data),label="train_accuracy")
plt.plot(np.array(test_correct)/len(train_data),label="test_accuracy")
plt.legend()

plt.show()
```



[132]:
```
x=100
im = inv_normalize(test_data[x][0])
plt.imshow(np.transpose(im.numpy(),(1,2,0)))

plt.show()
```



[133]:
```
test_data[x][0].shape
test_data[x][0].device
```

[133]: device(type='cpu')

[134]:
```
# Prediction for one of the samples.
model.eval()
with torch.no_grad():
    new_pred=model(to_device(test_data[x][0].view(1,3,224,224),device)).argmax()
print(f'Predicted value: {new_pred.item()} {class_names[new_pred.item()]}')
```

```
Predicted value: 0 daisy
```

##FLOPs In deep learning, FLOPs (Floating Point Operations) quantify the total number of arithmetic operations—such as additions, multiplications, and divisions—that a model performs during a single forward pass (i.e., when making a prediction). This metric serves as an indicator of a model's computational complexity. When discussing large-scale models, FLOPs are often expressed in GFLOPs (Giga Floating Point Operations), where 1 GFLOP equals one billion operations. This unit helps in comparing the computational demands of different models.

[90]:
```
# we use fvcore to calculate the FLOPs
!pip install fvcore
```

```
Requirement already satisfied: fvcore in c:\users\user\anaconda3\lib\site-
packages (0.1.5.post20221221)
Requirement already satisfied: numpy in c:\users\user\anaconda3\lib\site-
packages (from fvcore) (1.26.4)
Requirement already satisfied: yacs>=0.1.6 in c:\users\user\anaconda3\lib\site-
packages (from fvcore) (0.1.8)
Requirement already satisfied: pyyaml>=5.1 in c:\users\user\anaconda3\lib\site-
packages (from fvcore) (6.0.1)
Requirement already satisfied: tqdm in c:\users\user\anaconda3\lib\site-packages
(from fvcore) (4.66.5)
Requirement already satisfied: termcolor>=1.1 in
c:\users\user\anaconda3\lib\site-packages (from fvcore) (3.1.0)
Requirement already satisfied: Pillow in c:\users\user\anaconda3\lib\site-
packages (from fvcore) (10.4.0)
Requirement already satisfied: tabulate in c:\users\user\anaconda3\lib\site-
packages (from fvcore) (0.9.0)
Requirement already satisfied: iopath>=0.1.7 in
c:\users\user\anaconda3\lib\site-packages (from fvcore) (0.1.10)
Requirement already satisfied: typing-extensions in
c:\users\user\anaconda3\lib\site-packages (from iopath>=0.1.7->fvcore) (4.11.0)
Requirement already satisfied: portalocker in c:\users\user\anaconda3\lib\site-
packages (from iopath>=0.1.7->fvcore) (3.1.1)
Requirement already satisfied: colorama in c:\users\user\anaconda3\lib\site-
packages (from tqdm->fvcore) (0.4.6)
Requirement already satisfied: pywin32>=226 in c:\users\user\anaconda3\lib\site-
packages (from portalocker->iopath>=0.1.7->fvcore) (305.1)
```

[15]:
```
from fvcore.nn import FlopCountAnalysis
input = torch.randn(1, 3, 224, 224) # The input size should be the same as the␣
 ↪size that you put into your model
#Get the network and its FLOPs
num_classes = 5
model = TransferLearningModel(num_classes)
flops = FlopCountAnalysis(model, input)
print(f"FLOPs: {flops.total()/1e9:.5f} GFLOPs")
```

```
Unsupported operator aten::add_ encountered 58 time(s)
Unsupported operator aten::silu_ encountered 49 time(s)
Unsupported operator aten::sigmoid encountered 16 time(s)
Unsupported operator aten::mul encountered 25 time(s)
Unsupported operator aten::bernoulli_ encountered 9 time(s)
Unsupported operator aten::div_ encountered 9 time(s)
Unsupported operator aten::dropout_ encountered 1 time(s)
The following submodules of the model were never called during the trace of the
graph. They may be unused, or they were accessed by direct calls to .forward()
or via other python methods. In the latter case they will have zeros for
statistics, though their statistics will still contribute to their parent
calling module.
```

```
model.features.1.0.stochastic_depth, model.features.2.0.stochastic_depth,
model.features.3.0.stochastic_depth, model.features.4.0.stochastic_depth,
model.features.5.0.stochastic_depth, model.features.6.0.stochastic_depth,
model.features.7.0.stochastic_depth

FLOPs: 0.42060 GFLOPs
```